```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split as ttp
from sklearn.metrics import classification_report
import re
import string
import matplotlib.pyplot as plt

data=pd.read_csv('/content/drive/MyDrive/sentimental/file.csv')

data.head()
```

```
{"type":"dataframe","variable_name":"data"}
```

```
{"type":"dataframe","variable_name":"data_sent"}
```

```
{'type': 'dataframe', 'variable_name': 'data_sent'}
```

```
data.shape
```

```
(219294, 3)
```

```
data['labels'] = data['labels'].map({'neutral': 0, 'good':
1, 'bad': -1})

data.head()
```

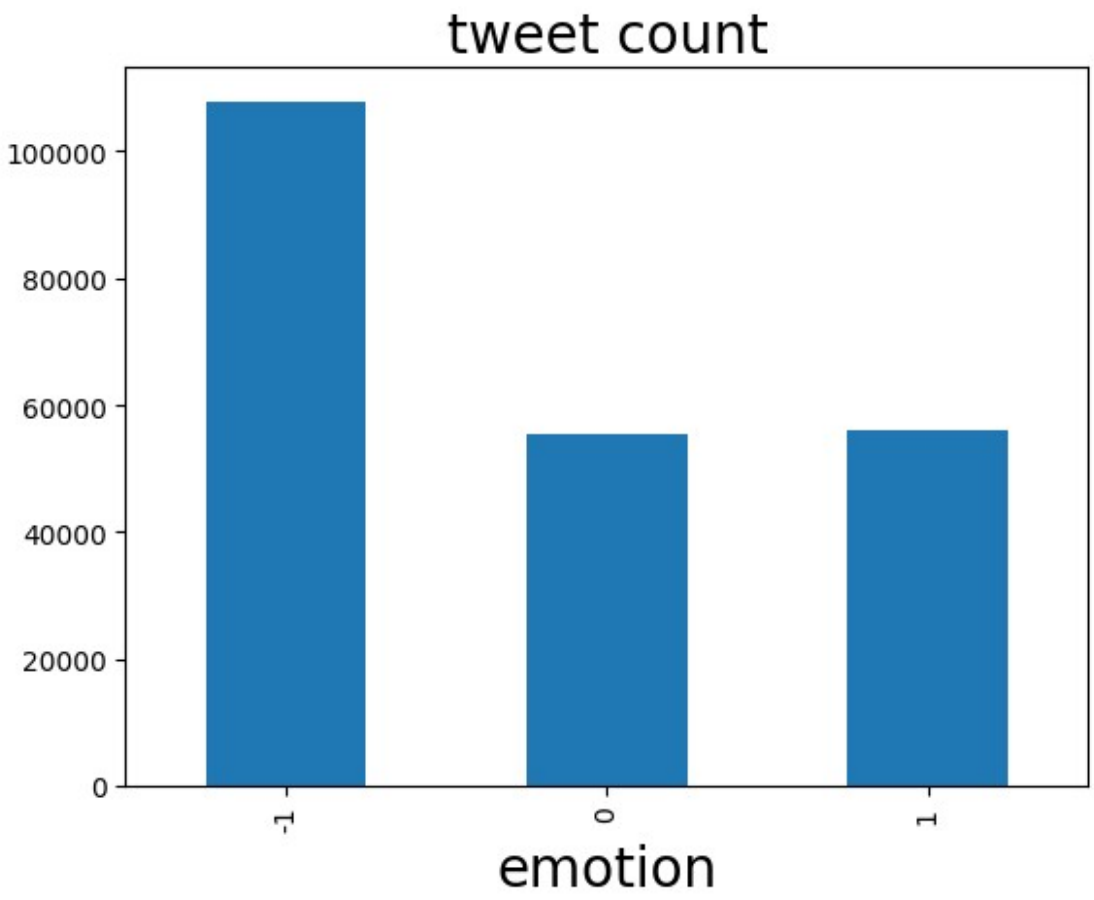
```
{"type":"dataframe","variable_name":"data"}
```

```
{"type":"dataframe","variable_name":"data_senti"}
```

```
{'type': 'dataframe', 'variable_name': 'data_senti'}
```

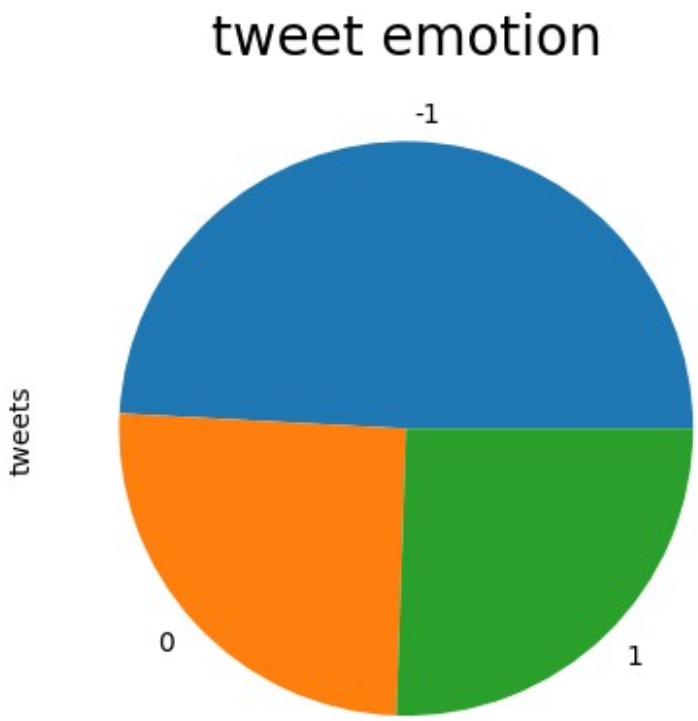
```
data_test=data.tail(10)
for i in range(219293,219284,-1):
 data.drop([i],axis=0,inplace=True)

data_test.to_csv("test.csv")

print(data.groupby(['labels'])['tweets'].count())
data.groupby(['labels'])['tweets'].count().plot(kind="bar")
plt.title("tweet count",size=20)
plt.xlabel("tweet",size=20)
plt.xlabel("emotion",size=20) #size=font size
plt.show()
```

```
labels
-1    107792
 0     55485
 1     56008
Name: tweets, dtype: int64
```




```
print(data.groupby(['labels'])['tweets'].count())
print("1= good\n -1= bad\n 0 =neutral")
data.groupby(['labels'])['tweets'].count().plot(kind="pie")
#data_merge.groupby(['class'])['text'].count().plot(kind="bar")
plt.title("tweet emotion",size=20)
plt.show()
```

```
labels
-1    107792
 0     55485
 1     56008
Name: tweets, dtype: int64
1= good
 -1= bad
 0 =neutral
```




```
data.isnull().sum()
```

```
Unnamed: 0    0
tweets        0
labels        0
dtype: int64
```

```
data=pd.DataFrame(data)
```

```
def filtering(data):
 text=data.lower()
 text=re.sub('\[.*?\]','',text)
 text=re.sub("\\W"," ",text)
 text=re.sub('https?://\S+|www\.\S+','',text)
 text=re.sub('<.*?>+','',text)
 text=re.sub('[%s]' % re.escape(string.punctuation),'',text)
 text=re.sub('\w*\d\w*','',text)
 return text
```

```
data["tweets"]=data["tweets"].apply(filtering)
data.head(10)
```


```
{"type":"dataframe","variable_name":"data"}
```



```
{"type":"dataframe","variable_name":"data"}
```

```
{'type': 'dataframe', 'variable_name': 'data'}
```

```
x=data["tweets"]
y=data["labels"]

x_train,x_test,y_train,y_test=ttp(x,y,test_size=0.25,random_state=0)

from sklearn.feature_extraction.text import TfidfVectorizer

vector=TfidfVectorizer()

xv_train=vector.fit_transform(x_train)

xv_test=vector.transform(x_test)

from sklearn.linear_model import LogisticRegression

LR=LogisticRegression()
LR.fit(xv_train,y_train)
```

```
LogisticRegression()
```

```
accuracy = LR.score(xv_test,y_test)
accuracy=round(accuracy,2)
print("Accuracy is: ",accuracy)
pred_LR=LR.predict(xv_test)
print(classification_report(y_test,pred_LR))
```

```
Accuracy is:  0.84
              precision    recall  f1-score   support

          -1       0.88      0.95      0.91     26831
           0       0.75      0.65      0.70     13921
           1       0.84      0.82      0.83     14070

    accuracy                           0.84     54822
   macro avg       0.82      0.81      0.81     54822
weighted avg       0.83      0.84      0.84     54822
```

```
def output_lable(n):
 if n==0:
  return "neutral"
 elif n==1:
  return "good"
 elif n==-1:
  return "bad"

def manual_testing(news):
 testing_news={"text":[news]}
 new_test=pd.DataFrame(testing_news)
 new_test["text"]=new_test["text"].apply(filtering)
```

```
 new_x_test=new_test["text"]
 news_xv_test=vector.transform(new_x_test)
 pred_LR=LR.predict (news_xv_test)
 print(output_lable(pred_LR[0]))
 return print("\n\nLRPrediction:{}".format(output_lable(pred_LR[0])))

news=str(input())
manual_testing(news)
```

```
1213hello
bad


LRPrediction:bad
```